HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Đỗ Bình Minh**
Ngày sinh	**: 24/12/2005**
Số điện thoại	**: 0329279863**
Mã sinh viên	**: B23DCCN542**
Lớp	**: D23CQCN10-B**
Nhóm lớp	**: 11**

Hà Nội – 2026

Mục lục

1. Tóm tắt các công việc đã thực hiện trong tuần 3
2. Chi tiết các công việc đã thực hiện trong tuần 3
2.1 Validate dữ liệu bên phía Frontend 3
2.1.1 Thư viện just-validate 3
2.1.2 Validate form đăng ký của ứng viên 3
2.1.3 Validate form đăng nhập của ứng viên 6
2.1.4 Validate form đăng ký của nhà tuyển dụng 7
2.1.5 Validate form đăng nhập của nhà tuyển dụng 8
2.2 Khởi tạo dự án bên phía Backend 10
2.2.1 Khởi tạo dự án bằng câu lệnh yarn 10
2.2.2 Cài đặt các thư viện chính 10
3. Kết quả đạt được 11
4. Khó khăn gặp phải 11
5. Hướng xử lý 12
6. Kế hoạch tuần tiếp theo 12

1. Tóm tắt các công việc đã thực hiện trong tuần

Sau khi đã hoàn thiện website về mặt giao diện trong tuần trước. Trong tuần này, công việc trọng tâm của em là xử lý validate các form đăng ký, đăng nhập bên phía Frontend. Bên cạnh đó, khởi tạo dự án website bên phía backend. Mục tiêu đặt ra là đảm bảo dữ liệu đầu vào hợp lệ, thiết lập môi trường backend ban đầu và tạo nền tảng để phát triển các API xác thực người dùng trong các giai đoạn tiếp theo.

2. Chi tiết các công việc đã thực hiện trong tuần

2.1 Validate dữ liệu bên phía Frontend

2.1.1 Thư viện just-validate

- Just-validate là một thư viện JavaScript dùng để **kiểm tra dữ liệu form ở phía client**.

- Khi người dùng nhập form đăng ký, đăng nhập, liên hệ...
- just-validate sẽ giúp kiểm tra xem:
    - ô input có bị bỏ trống không
    - email có đúng định dạng không
    - mật khẩu có đủ độ dài không
    - xác nhận mật khẩu có khớp không

- Cách cài đặt:

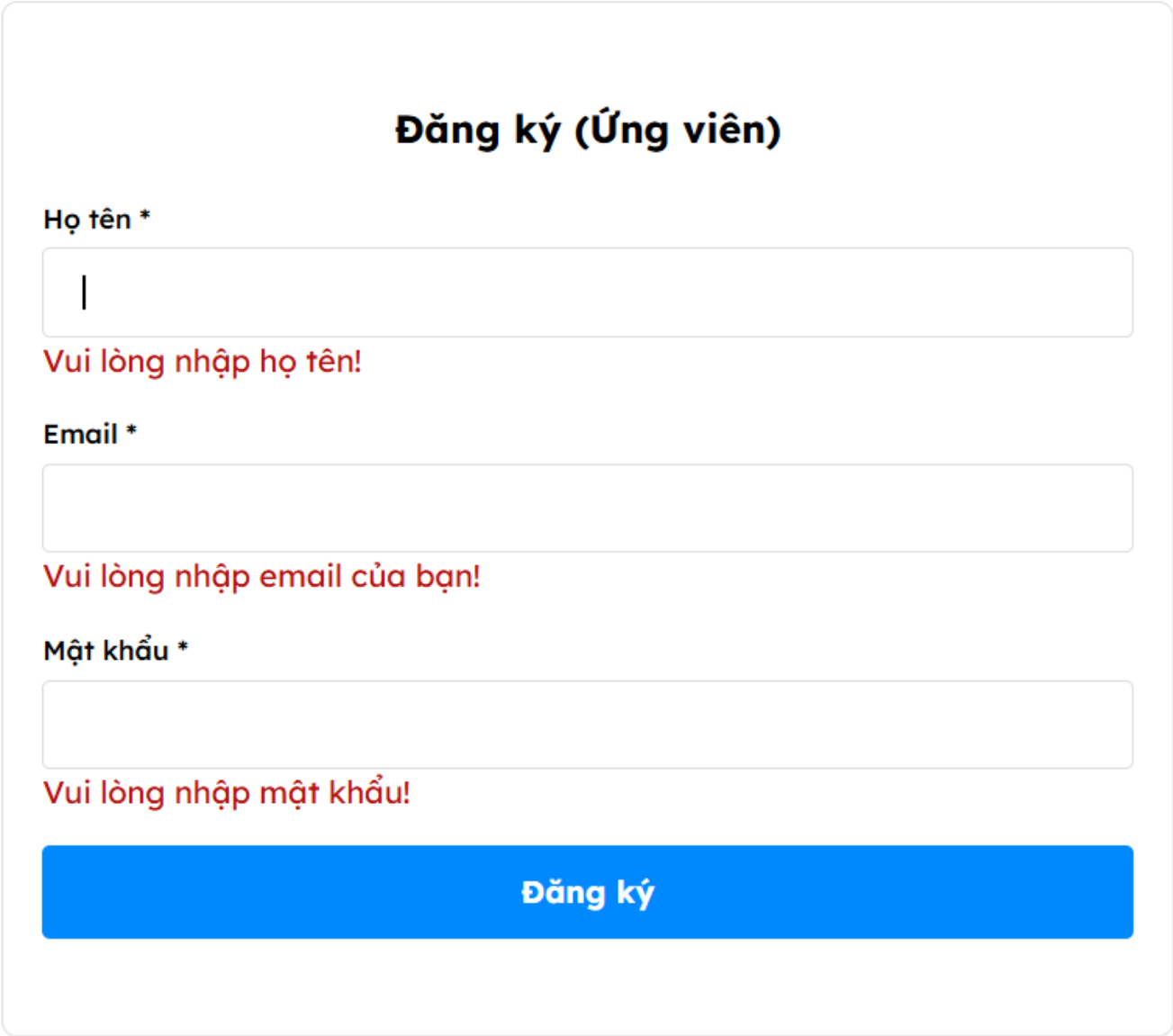
- Mở terminal của dự án phía Frontend trên VS code
- Gõ lệnh yarn init để khởi tạo file package.json giúp yarn quản lý các thư viện
- Gõ lệnh yarn add just-validate để cài đặt thư viện vào dự án

2.1.2 Validate form đăng ký của ứng viên

Logic xử lý:

```
validator
  .addField('#fullName', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập họ tên!'
    },
    {
      rule: 'minLength',
      value: 5,
      errorMessage: 'Họ tên phải có ít nhất 5 ký tự!',
    },
    {
      rule: 'maxLength',
      value: 50,
      errorMessage: 'Họ tên không được vượt quá 50 ký tự!',
    },
  ])
  .addField('#email', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập email của bạn!',
    },
    {
      rule: 'email',
      errorMessage: 'Email không đúng định dạng!',
    },
  ])
  .addField('#password', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập mật khẩu!',
    },
    {
      validator: (value: string) => value.length >= 8,
      errorMessage: 'Mật khẩu phải chứa ít nhất 8 ký tự!',
    },
    {
      validator: (value: string) => /[A-Z]/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một chữ cái in hoa!',
    },
    {
      validator: (value: string) => /[a-z]/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một chữ cái thường!',
    },
    {
      validator: (value: string) => /\d/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một chữ số!',
    },
    {
      validator: (value: string) => /[@$!%*?&]/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một ký tự đặc biệt!',
    },
  ])
```

Kết quả:


Đăng ký (Ứng viên)
Họ tên *
Vui lòng nhập họ tên!
Email *
Vui lòng nhập email của bạn!
Mật khẩu *
Vui lòng nhập mật khẩu!
Đăng ký

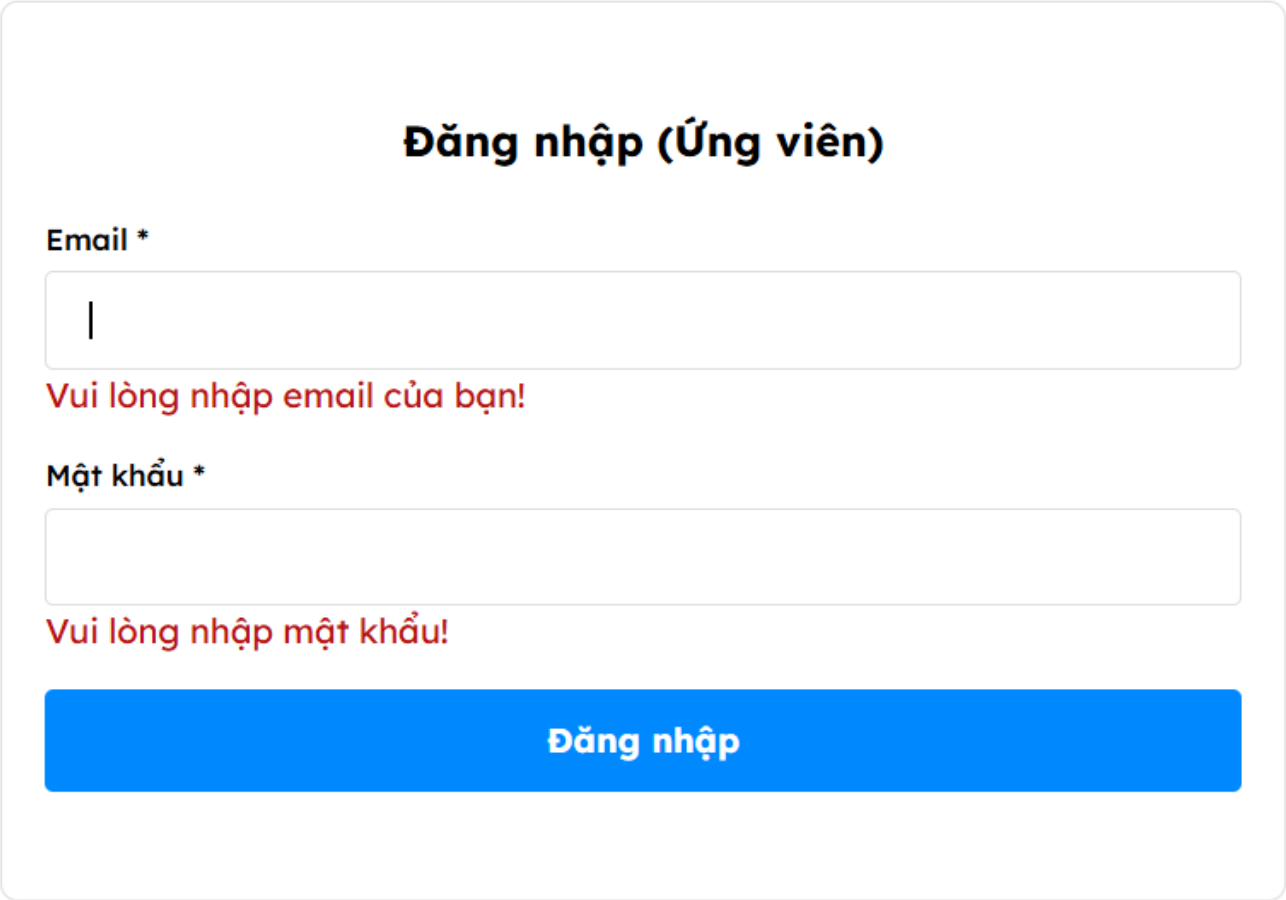
2.1.3 Validate form đăng nhập của ứng viên

Logic xử lý:

```
const validator = new JustValidate("#loginForm");

validator
  .addField('#email', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập email của bạn!',
    },
    {
      rule: 'email',
      errorMessage: 'Email không đúng định dạng!',
    },
  ])
  .addField('#password', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập mật khẩu!',
    },
  ])
```

Kết quả:

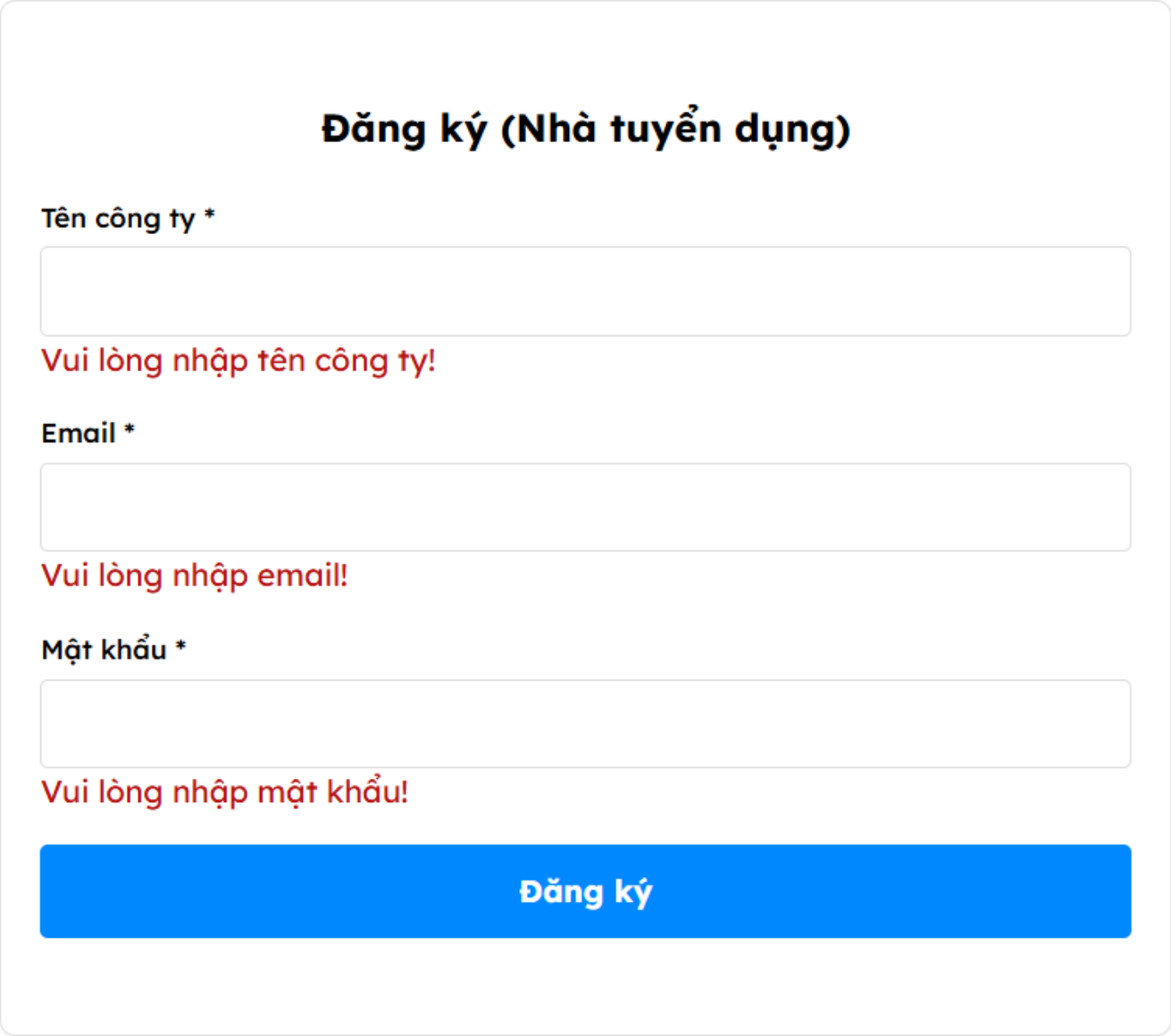


2.1.4 Validate form đăng ký của nhà tuyển dụng

Logic xử lý:

```
validator
  .addField('#companyName', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập tên công ty!'
    },
    {
      rule: 'maxLength',
      value: 200,
      errorMessage: 'Tên công ty không được vượt quá 200 ký tự!',
    },
  ])
  .addField('#email', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập email!',
    },
    {
      rule: 'email',
      errorMessage: 'Email không đúng định dạng!',
    },
  ])
  .addField('#password', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập mật khẩu!',
    },
    {
      validator: (value: string) => value.length >= 8,
      errorMessage: 'Mật khẩu phải chứa ít nhất 8 ký tự!',
    },
    {
      validator: (value: string) => /[A-Z]/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một chữ cái in hoa!',
    },
    {
      validator: (value: string) => /[a-z]/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một chữ cái thường!',
    },
    {
      validator: (value: string) => /\d/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một chữ số!',
    },
    {
      validator: (value: string) => /[@$!%*?&]/.test(value),
      errorMessage: 'Mật khẩu phải chứa ít nhất một ký tự đặc biệt!',
    },
  ])
```

Kết quả:

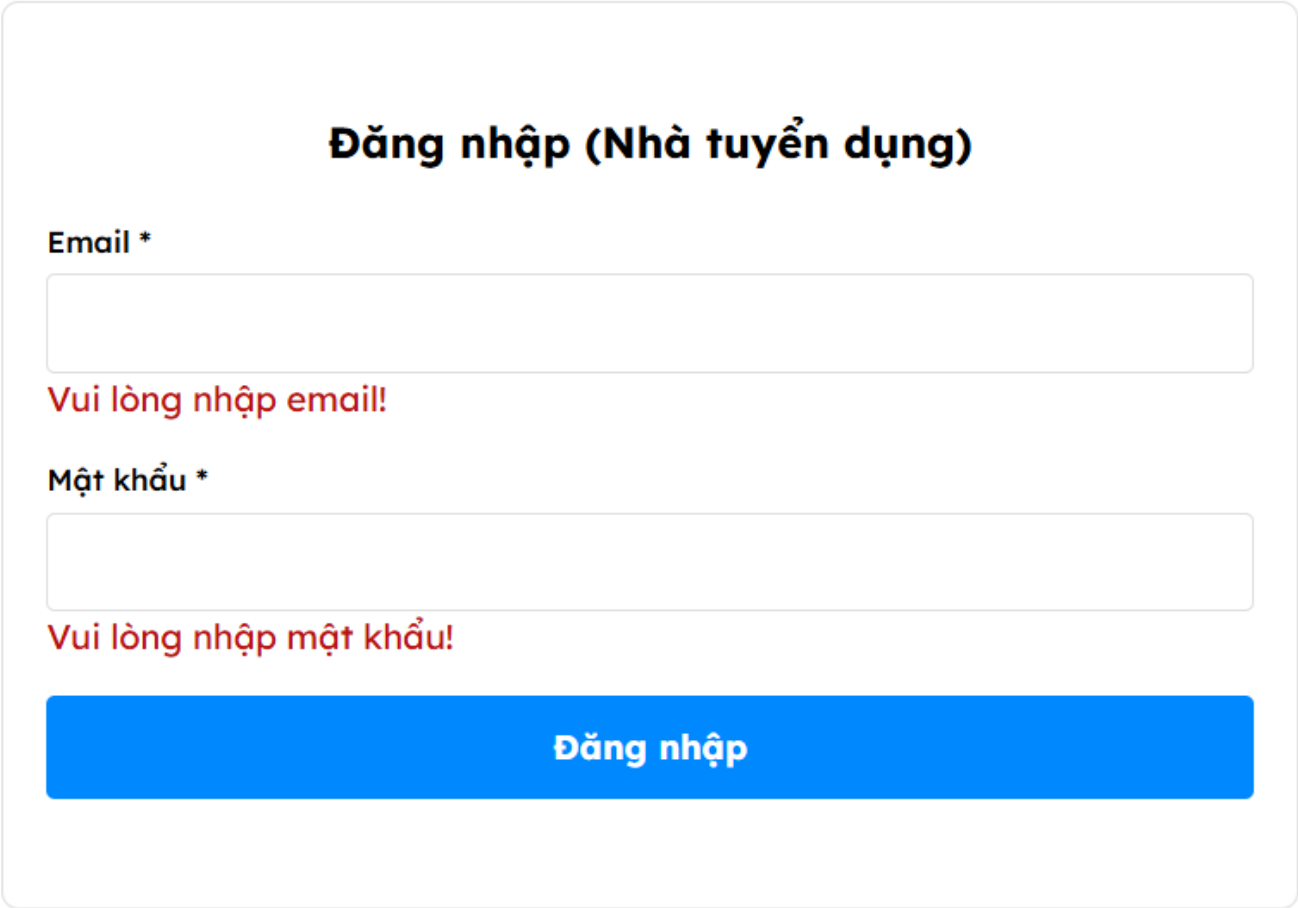


2.1.5 Validate form đăng nhập của nhà tuyển dụng

Logic xử lý:

```
validator
  .addField('#email', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập email!',
    },
    {
      rule: 'email',
      errorMessage: 'Email không đúng định dạng!',
    },
  ])
  .addField('#password', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập mật khẩu!',
    },
  ])
```

Kết quả:



2.2 Khởi tạo dự án bên phía Backend

2.2.1 Khởi tạo dự án bằng câu lệnh yarn

Mở terminal trên VS code nhập lệnh: yarn init

Kết quả:

- Tạo file package.json
- Khai báo thông tin cơ bản của project

2.2.2 Cài đặt các thư viện chính

Express (Framework backend)

Câu lệnh cài đặt: yarn add express

Vai trò:

- Xây dựng RESTful API
- Xử lý routing, middleware

TypeScript

Câu lệnh cài đặt: yarn add -D typescript

Vai trò:

- Viết backend bằng TypeScript thay vì JavaScript
- Tăng tính an toàn kiểu dữ liệu

ts-node

Câu lệnh cài đặt: yarn add -D ts-node

Vai trò:

- Chạy trực tiếp file .ts mà không cần build

Cài kiểu dữ liệu Node.js

Câu lệnh cài đặt: yarn add -D @types/node

Vai trò:

- Hỗ trợ TypeScript hiểu các API của Node.js

Cài kiểu dữ liệu Express

Câu lệnh cài đặt: yarn add -D @types/express

Vai trò:

- Cung cấp type cho Express khi dùng TypeScript

Nodemon

Câu lệnh cài dặt: yarn add -D nodemon

Vai trò:

- Tự động restart server khi code thay đổi

3. Kết quả đạt được

- Hoàn thiện chức năng validate dữ liệu cho các form đăng ký và đăng nhập ở phía Frontend, đảm bảo kiểm tra đầy đủ các trường thông tin như email, mật khẩu, xác nhận mật khẩu.

- Cải thiện trải nghiệm người dùng khi nhập liệu thông qua việc hiển thị thông báo lỗi rõ ràng và kịp thời.

- Khởi tạo thành công dự án Backend bằng Node.js kết hợp với Express và TypeScript.

Cài đặt đầy đủ các thư viện cần thiết phục vụ cho quá trình phát triển Backend.

- Thiết lập môi trường chạy server cơ bản và bước đầu chạy thử thành công với ts-node và nodemon.

- Tạo nền tảng ban đầu để phát triển các API trong các giai đoạn tiếp theo của dự án.

4. Khó khăn gặp phải

- Lỗi khi chạy file TypeScript với Node.js do chưa cấu hình đúng môi trường (Unknown file extension ".ts").

- Chưa quen với việc cấu hình TypeScript trong môi trường Backend nên mất thời gian tìm hiểu.

- Việc xử lý validate form cần kiểm tra nhiều trường hợp khác nhau, dễ xảy ra thiếu sót logic.

- Khó khăn trong việc đảm bảo đồng bộ giữa validate phía Frontend và định hướng xử lý phía Backend.

5. Hướng xử lý

- Tra cứu tài liệu chính thức và sửa lại tên package đúng chuẩn khi cài đặt thư viện.
- Sử dụng ts-node kết hợp với nodemon để chạy trực tiếp file TypeScript mà không cần build.
- Bổ sung các thư viện type (@types/node, @types/express) để hỗ trợ TypeScript hoạt động ổn định.
- Xây dựng logic validate rõ ràng theo từng trường hợp (rỗng, sai định dạng, không khớp...).
- Tham khảo tài liệu và ví dụ thực tế để hoàn thiện cấu hình môi trường Backend.
- Kiểm tra và test lại nhiều lần để đảm bảo các form hoạt động đúng yêu cầu.

6. Kế hoạch tuần tiếp theo

- Xây dựng luồng xử lý dữ liệu hoàn chỉnh giữa Frontend và Backend, bắt đầu từ việc gửi dữ liệu từ phía Frontend lên server.
- Triển khai các API phía Backend để tiếp nhận và xử lý dữ liệu từ người dùng.
- Thiết kế và tạo cơ sở dữ liệu, đồng thời thiết lập kết nối giữa Backend và hệ quản trị cơ sở dữ liệu.
- Xây dựng các model dữ liệu (đặc biệt là bảng accounts/user) phục vụ cho chức năng xác thực.
- Thực hiện lưu trữ dữ liệu người dùng vào cơ sở dữ liệu thông qua các API đã xây dựng.
- Tích hợp và thử nghiệm API bằng công cụ hỗ trợ (như Postman hoặc Joi để validate phía Backend).
- Hoàn thiện việc nhận phản hồi từ Backend và xử lý hiển thị kết quả ở phía Frontend